As the string example shows, you can specify as much or as little of the string as you like. The same applies to arrays; you can check for a sequence of elements at the beginning or end:

11-matchers-included-in-rspec-expectations/02/higher_order_matchers.rb
```
expect([1, 2, 3]).to start_with(1, 2)
expect([1, 2, 3]).to end_with(2, 3)
```

The same caution about using the Ruby splat operator with include applies to starts_with and ends_with as well.

Following the aliasing pattern we've seen elsewhere, these matchers have two aliases each:

- a_string_starting_with / a_string_ending_with
- a_collection_starting_with / a_collection_ending_with

You could combine these, for example:

11-matchers-included-in-rspec-expectations/02/higher_order_matchers.rb
```
expect(['list', 'of', 'words']).to start_with(
  a_string_ending_with('st')
).and end_with(
  a_string_starting_with('wo')
)
```

The outer start_with matcher checks the word 'list' using the inner a_string_ending_with, and so on.

all

The all matcher is somewhat of an oddity: it is the only built-in matcher that is not a verb, and it is the only one that *always* takes another matcher as an argument:

11-matchers-included-in-rspec-expectations/02/higher_order_matchers.rb
```
numbers = [2, 4, 6, 8]
expect(numbers).to all be_even
```

This expression does exactly what it says: it expects all the numbers in the array to be even. Here, 'be_even' is a dynamic predicate like the ones we saw in *Dynamic Predicates*, on page 194. It calls 'even?' on each element of the array.

One gotcha to be aware of is that, like Enumerable#all?, this matcher passes against an empty array. This can lead to surprises. Consider the following incorrect method for generating a list of numbers: